Email Notification Scheduler

Design Document — Scheduler Service (scheduler-service module)

All 3 DB Strategies · ShedLock · Kafka · Spring @Scheduled

Version 1.0 | Feb 2026 | Spring Boot 3.2 + Java 17 + PostgreSQL 15

1. Overview

The **Email Notification Scheduler** is a background job inside **scheduler-service** that periodically scans for recurring transaction schedules due for notification, sends email and push alerts via Kafka, then **updates the next notification date** in the database so the cycle continues at the correct frequency (Weekly / Bi-Weekly / Monthly).

Design Goals

Goal	Decision
Single source of truth	All 3 DB strategies kept in sync — each serves a different query pattern
No duplicate sends	ShedLock prevents concurrent job runs across pods; Redis idempotency key per run
Failure resilience	Per-channel failure counters; auto-deactivate after threshold; full audit log
Extensibility	Template code column → easy to add new notification types (TXN_FAILED, RECEIVER_UPDATED)
Observability	Micrometer counters on dispatched/failed; Prometheus scrape; structured log per strategy

2. Database Strategies

Three complementary table approaches are used together. Each serves a distinct purpose. The job runs all three strategies in a single cron invocation.

#	Table	Key Column	Purpose	Updated A
S1	recurring_schedule	next_notification_at	Fast index scan — primary cron driver	YES → next_notification_at = nov
S2	notification_schedule	next_send_at	Per-channel (EMAIL/PUSH) independent control	YES → failure tracking
S3	notification_log	next_send_at	Immutable audit trail — every send appends a new row (never updated)	N/A (append-only log)

Strategy 1 — recurring_schedule.next_notification_at

New columns added via Flyway V2 migration:

- **next_notification_at** TIMESTAMP — when to next fire a notification
- **last_notification_at** TIMESTAMP — last successful send time (audit)
- **notification_attempts** INT — consecutive failure counter (reset on success)

Index: idx_schedule_next_notif ON (next_notification_at, status) WHERE status='ACTIVE'
Query: SELECT ... WHERE status='ACTIVE' AND next_notification_at <= NOW()
Update after send: UPDATE recurring_schedule SET next_notification_at = NOW() + freq, last_notification_at = NOW(), notification_attempts = 0 WHERE id = ?

Strategy 2 — notification_schedule (dedicated control table)

New table with **one row per (schedule_id, channel)**. EMAIL and PUSH are tracked independently — user can disable email but keep push.

Key columns: **next_send_at**, **last_sent_at**, **last_status**, **failure_count**, **max_retry**, **is_active**
Index: idx_notif_sched_next_send ON (next_send_at, is_active) WHERE is_active=TRUE
Update after send: UPDATE notification_schedule SET next_send_at=?, last_sent_at=?, last_status='SENT', failure_count=0 WHERE id=?
Deactivate on failure: UPDATE notification_schedule SET is_active=false, last_status='SUPPRESSED' WHERE id=?

Strategy 3 — notification_log.next_send_at (audit trail)

Every notification dispatch **appends a new row** to notification_log. The row carries **next_send_at** = computed next date at time of dispatch. This gives a full, immutable progression: you can see every date shift over time.

This table is never updated — only inserted. Failed sends get their own row with status=FAILED. Retries append another row. This means the log shows: SENT → FAILED → SENT → SENT...

New columns added: next_send_at, channel, template_code, retry_count, error_message
Index: idx_notif_log_next_send ON (next_send_at, status) WHERE status IN ('PENDING','FAILED')

3. Scheduler Job Flow

3.1 Trigger Mechanisms

Trigger	Expression	Description
Fixed cron (dev)	0 * * * * * (every minute)	Scans all 3 strategies each minute
Fixed cron (prod)	0 0 9 * * * (daily 9 AM)	Override via scheduler.notification.cron property
DB-driven (S1)	next_notification_at <= NOW()	Records drive timing; cron just scans
DB-driven (S2)	next_send_at <= NOW()	Per-channel independent timing
Retry (S3)	FAILED rows, retryCount < 3	Retries on every job run

3.2 Single Job Run — Step by Step

Step	Action	Table Written
1	Cron fires → ShedLock acquires distributed lock on shedlock table	shedlock
2	processStrategy1(now): query recurring_schedule WHERE next_notification_at <= NOW()	
3	For each row: dispatch EMAIL (if emailNotify=true), dispatch PUSH (if pushNotify=true)	notification_log (S3 row)
4	Success: UPDATE recurring_schedule SET next_notification_at=NOW()+frequency, notification_attempts=0	recurring_schedule
5	Failure: incrementFailureCount; if count >= 3 → log warn and skip next cycle	recurring_schedule
6	processStrategy2(now): query notification_schedule WHERE next_send_at <= NOW()	
7	For each row: dispatch via Kafka	notification_log (S3 row)
8	Success: UPDATE notification_schedule SET next_send_at=lastSentAt+frequency, failure_count=0	notification_schedule
9	Failure: markFailed; if failure_count >= max_retry → deactivate row	notification_schedule
10	processRetryStrategy3(): find FAILED log rows with retryCount < 3, re-dispatch	notification_log
11	ShedLock released; next pod can acquire on next minute	shedlock

3.3 next_send_at Computation

Frequency	Formula	Example (sent 2026-02-28 09:00)
WEEKLY	lastSentAt + 7 days	2026-03-07 09:00
BIWEEKLY	lastSentAt + 14 days	2026-03-14 09:00
MONTHLY	lastSentAt + 1 month	2026-03-28 09:00

Computed by **NextSendDateCalculator** — single shared utility used by all 3 strategies.

4. Concurrency & Safety

4.1 ShedLock — Distributed Lock

Problem: Multiple scheduler-service pods run in Kubernetes. Without coordination, all pods would fire the job simultaneously → duplicate emails sent.

Solution: ShedLock acquires a row-level lock in the **shedlock** PostgreSQL table. Only the pod that wins the lock executes the job. Others skip silently.

Config: lockAtLeastFor=PT50S, lockAtMostFor=PT55S (prevents ghost locks if pod crashes). The DB clock is used (**usingDbTime()**) so JVM clock skew across pods is irrelevant.

4.2 Optimistic Locking

recurring_schedule has a **version** column (JPA @Version). If two pods somehow read the same row concurrently (before ShedLock fires), the UPDATE will return rowCount=0 for the loser — logged as a warning, the row will be picked up on the next cron tick.

notification_schedule also has a **version** column for the same protection.

4.3 Idempotency

Before publishing to Kafka, the job checks a Redis key:
Key: **notif:sent:{scheduleId}:{channel}:{YYYYMMDD}** — TTL 25 hours.
If the key exists, the dispatch is skipped (SKIPPED status logged in notification_log).
This prevents double-send if the same job somehow runs twice (e.g. during deployment).

5. Failure Handling

Scenario	Detection	Action	Recovery
Kafka publish fails	Exception in dispatchService.dispatch()	dispatchFailureCount (S1); markFailed() log FAILED (S2)	Retry (S2) next run (S1 max 3 tries)
DB update fails (next_send_at)	Exception in markNotificationSent()	Optimistic lock conflict logged; job continues	Next run scan picks it up again
Max retries exceeded (S2)	failureCount >= maxRetry	deactivate() sets is_active=false, last_status=DEACTIVATED	Manual reactivation via API or ops tooling
Max retries exceeded (S1)	notificationAttempts >= 3	Warning logged; row skipped in that run	Ops alert on Prometheus counter; manual fix
Pod crash mid-job	ShedLock lock_until expires after PT55S	Next pod re-acquires lock and re-runs job	Idempotency key prevents double send
Email SMTP down	Exception from NotificationDispatchService	KafkaEvent published; notification-service handles SMTP	SMTP handles backoff; Kafka preserves retries

6. Class & Table Summary

Class / File	Layer	Responsibility
EmailNotificationSchedulerJob	Job	Main cron entry; orchestrates all 3 strategies; ShedLock
NotificationDispatchService	Service	Publish Kafka event; persist NotificationLog (S3) row
NotificationScheduleService	Service	Bootstrap S2 rows; toggle channels; handle freq changes
NextSendDateCalculator	Utility	Single place to compute next notification date per frequency
RecurringScheduleRef	Entity	JPA entity for recurring_schedule (S1 columns only)
NotificationSchedule	Entity	JPA entity for notification_schedule table (S2)
NotificationLog	Entity	JPA entity for notification_log with next_send_at (S3)
RecurringScheduleRefRepository	Repo	findDueForNotification; markNotificationSent; incrementFailureCount
NotificationScheduleRepository	Repo	findDueNotifications; markSentAndAdvance; markFailed; deactivate
NotificationLogRepository	Repo	findLatestSent; findPendingRetries
SchedulerConfig	Config	ShedLock LockProvider; @EnableScheduling; @EnableSchedulerLock
V2__notification_scheduler_tables.sql	Migration	All DDL: new columns (S1), new table (S2), updated table (S3)

Kafka Topic

Topic	Producer	Consumer	Payload
notification.send	NotificationDispatchService	Notification Service (email + push workers)	{scheduleId, userId, channel, templateCode, mtc

7. Observability

Metric / Log	Type	Description
notif.dispatched.count{strategy,channel}	Counter	Incremented on each successful dispatch
notif.failed.count{strategy,channel}	Counter	Incremented on each dispatch failure
notif.job.duration.ms	Timer	Full job run duration
notif.deactivated.count	Counter	Rows deactivated in notification_schedule (S2)
Log: [Strategy-1] next_notification_at updated to ...	INFO	Per-schedule update confirmation
Log: [Strategy-2] notification_schedule.next_send_at set to .	INFO	Per-row S2 update
Log: Optimistic lock conflict on scheduleId=...	WARN	Concurrent access detected — not an error
Log: Max failures reached for scheduleId=...	WARN	Action needed: ops should investigate

Summary

The notification scheduler is self-contained inside **scheduler-service** and:

• Uses **3 complementary DB strategies** — each addresses a different concern (fast scan, per-channel control, full audit trail)

• Runs on **two triggers simultaneously**: a fixed cron AND database-driven next_send_at / next_notification_at timestamps

• **Updates the next date atomically** after every successful dispatch using @Modifying JPQL queries with optimistic locking

• Is **cluster-safe** via ShedLock on PostgreSQL; no duplicate sends possible

• **Full failure handling**: per-channel failure counters, auto-deactivation, Strategy-3 retry loop, idempotency keys in Redis

• All sends go through **Kafka topic notification.send** — decoupled from SMTP/FCM